Features of Spring Framework

•Flexible in nature (versatile application framework)
•Non-invasive application development framework Page

1. Flexible in Nature

Also called: versatile application framework.

Idea: Spring is flexible in nature — it can be used alongside almost any other technology or framework, fitting into the layer where it is needed, without forcing you to throw away your existing work.

This short story explains the idea of a 'flexible / versatile application framework' using a real-world style scenario between a company using Struts and the makers of Spring.

A company already working with Struts
There is a company that is using the Struts framework for developing their project, or the project is already ongoing on Struts.

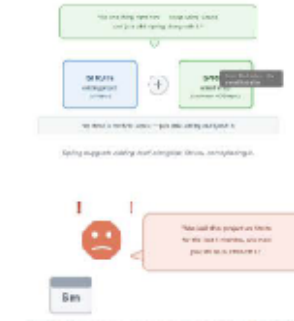
Spring framework enters the market
Just then, the Spring framework arrives in the market with its full set of features.
Page.

Spring's makers market their framework
So the makers of Spring framework thought,
"let's do some marketing with our framework,
so that it becomes popular." They did this by:

- Releasing it in the market for free, as open source
- Conducting webinars
- Providing documentation and support to customers

The company takes notice
One fine day, the company that is using the Struts framework sees all this and thinks, "let's see what Spring framework is, and whether it is helpful or not." So they conduct a meeting with the Spring framework people and ask what advantage they would get by using Spring.

Spring's suggestion
The Spring framework people replied:
"Do one thing right now — as you are already using the Struts framework, in addition to it, use our Spring framework along with it."



The resolution — use both together
The Spring people then clarified:
"No — don't remove Struts. Just use Spring framework along with Struts."
The company agreed, and they decided not to remove anything but to use each framework in the layer or tier where it fits best.
Slowly, the framework worked effectively and became popular, because it worked well in every layer of the application.

2. Non-Invasive Framework (Spring)

The Spring Framework is called a non-invasive framework.
This means:
- While using Spring, we do not need to use Spring-specific classes in our main application logic.
- Our code remains independent and clean.

Why is this useful?
Because of this:
- Our application is not tightly connected to Spring.
- If in the future we want to switch to another framework, it is very easy to do so.

What do we need to do to switch?
- Remove the Spring libraries (JAR files)
- Connect (plug) the same code with another framework
  We do not need to rewrite or delete our existing logic.

Types of Classes in Java

When we build an application, we write many classes.
Based on how we write them, we can divide them into 3 types:

1. POJO (Plain Old Java Object)

A POJO is a simple Java class:

It does not use any external libraries
It does not extend any special class
It is just a normal Java class

In simple words:
A class that works independently without any framework is called POJO.

Example:

```
public class Student {
   String name;
   int age;

   void display() {
      System.out.println(name + " " + age);
   }
}
```

This is a POJO because:

No external library used
No special inheritance

2. Java Bean

Be a public class
Have a public no-argument (default) constructor
Have private properties with getter/setter methods
Implement Serializable (so its state can be saved and restored)

Example

```
import java.io.Serializable;

public class Student implements Serializable {

   private String name;
   private int age;

   public Student() {
   }

   public String getName() {
      return name;
   }

   public void setName(String name) {
      this.name = name;
   }

   public int getAge() {
      return age;
   }

   public void setAge(int age) {
      this.age = age;
   }
}
```

3. Bean / Component

A Bean (or Component) is a class that:

Contains methods with logic
Performs some operations or tasks

In simple words:
A class that does actual work (business logic) is called a Bean or Component.

What is "arbitrary set of methods"?

"Arbitrary" means
Any kind of methods based on your need

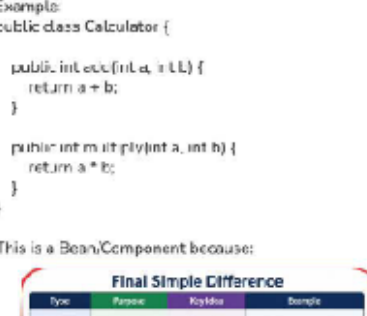
So:

There is no fixed rule
You can write any methods with any logic

Example:

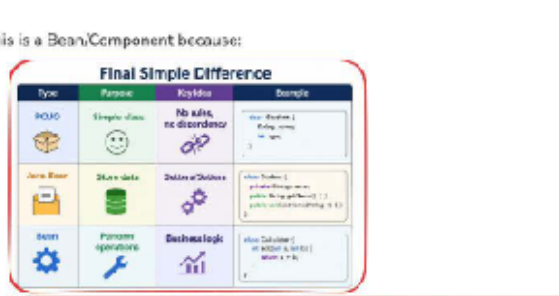
```
public class Calculator {

   public int add(int a, int b) {
      return a + b;
   }

   public int multiply(int a, int b) {
      return a * b;
   }
}
```

This is a Bean/Component because:



Final Simple Difference

Type	Purpose	Key idea	Example
POJO	Simple class	No rules, no dependency	[illegible]
Java Bean	Store data	[illegible]	[illegible]
Bean	Perform operations	Business logic	[illegible]



When to Use POJO, Java Bean and Bean/Component?

	1. POJO (Plain Old Java Object)	2. Java Bean	3. Bean / Component
Purpose	[illegible]	[illegible]	[illegible]
When to Use	[illegible]	[illegible]	[illegible]

Why Do We Need Spring Core?

Problem with Traditional Java Classes

A Component class often cannot complete its work on its own.
It usually needs help from another class. Such a class is called a **dependent class**.

Example

```
class EmailService {
   public void sendEmail() {
      System.out.println("Email Sent");
   }
}

class UserService {
   private EmailService emailService = new EmailService();

   public void registerUser() {
      System.out.println("User Registered");
      emailService.sendEmail();
   }
}
```

Here, `UserService` depends on `EmailService`.

What is the Problem?

Suppose tomorrow the company decides to replace `EmailService` with `WhatsAppService`.
Then we have to modify the code inside `UserService`.

```
private WhatsAppService whatsAppService = new WhatsAppService();
```

If hundreds of classes depend on `EmailService`,
we must update all these classes.

This requires:

* More effort
* More time
* More maintenance
* Higher chance of bugs

This problem is called **tight coupling**.

How Spring Solves This Problem

Spring Core says:

> "Don't worry about creating dependent objects. Give that responsibility to me."
> Instead of classes creating their own dependencies, Spring creates and manages them.

Feature of spring framework

|- Versatile in nature [use some common word in the place of Versatile where ever you see]
or you can say as versatile application framework

|- Non invasive application development framework

|- Versatile in nature

---

or you can say as versatile application framework

|- Create one story with the image like one company is there who are using the struts framework for developing the project or in ongoing project and spring framework just came in market with it's full feature
|- so the maker of spring framework thought let's do some marketing with our framework so that it will become popular
|- so they released in the market for free or as open source
|- conducting through wenar
|- providing documentation support to the customers

|- One fine day the one company who is using the struts framework had seen these thing and they thought let's see what spring framework is and is it helpful or not so they conduct meeting with the people of spring framework and ask what when can get the advantage by using your framework

|- Spring framework people repied "do one thing right now as you are using struts framework addition with it use our spring framework in it."

|- Company people get surprised and scolled the framework people that from last 6 month we had builded the project on struts and now you are telling me to remove struts and use spring framework

|- Then spring people replied no do one thing use spring framework along with struts

|- They agreed and they decide to not use all let's use in some layer or in some tier
|- and they slowly the framework worked effectively and get popular because it's worked in every layer of application

|- Non invasive application development framework

If you are already working on a project using any technology and want to use the Spring Framework, you do not need to delete or rebuild your existing work.
Spring can easily fit into your current system and help you improve it by filling the missing parts.

Example:

When we build applications using the Apache Struts,
it mainly supports only the presentation layer (UI part).

But what about:
Business logic
Database operations (persistence layer)

This is where Spring helps.

We can combine Spring with Struts, so:

Struts handles the UI
Spring handles business and database logic

This makes the entire application stronger and more complete.

If you are already working on a project using any technology and want to use the Spring Framework, you do not need to delete or rebuild your existing work.

Spring can easily fit into your current system and help you improve it by filling the missing parts.

Example:

When we build applications using the Apache Struts, it mainly supports only the presentation layer (UI part).

But what about:

Business logic
Database operations (persistence layer)

This is where Spring helps.

We can combine Spring with Struts, so:

Struts handles the UI
Spring handles business and database logic

This makes the entire application stronger and more complete.

This ability to work with other technologies easily is one of the main reasons why Spring became popular very quickly.

Non-Invasive Framework (Spring)

The Spring Framework is called a non-invasive framework.

This means:

While using Spring, we do not need to use Spring-specific classes in our main application logic.
Our code remains independent and clean.
Why is this useful?

Because of this:

Our application is not tightly connected to Spring.
If in the future we want to switch to another framework, it is very easy.

What do we need to do to switch?
Remove Spring libraries (JAR files)
Connect (plug) the same code with another framework

We do not need to rewrite or delete our existing logic.

In simple words:
Spring lets you build applications in a way that your code:
> stays flexible
> is easy to change
> is not dependent on one framework

Spring Framework
1 Multiple Types of application development types
2 it supports end to end applications
3 Non-invasive application development framework
4 Versatile application development framework

Types of Classes in Java

When we build an application, we write many classes. Based on how we write them, we can divide them into 3 types:

1. POJO (Plain Old Java Object)

A POJO is a simple Java class:

It does not use any external libraries
It does not extend any special class
It is just a normal Java class

In easy words:
A class that works independently without any framework is called POJO.

Example:

```
public class Student {
   String name
   int age

   void display() {
      System.out.println(name + " " + age);
   }
}
```

This is a POJO because:

2.Java Bean

A Java Bean is a special type of class with some rules:

Rules:
Class should be public
Must have a default constructor
Variables should be private
Must have getter and setter methods

Summary :
A class used to store data properly is called a Java Bean.

Example:

```
public class Student {
   private String name;
   private int age;

   public Student() {
   }

   public String getName() {
      return name;
   }

   public void setName(String name) {
      this.name = name;
   }

   public int getAge() {
      return age;
   }

   public void setAge(int age) {
      this.age = age;
   }
}
```

This is a Java Bean because:

3. Bean / Component

A Bean (or Component) is a class that:

Contains methods with logic
Performs some operations or tasks

In simple words:
A class that does actual work (business logic) is called a Bean or Component

❓ What is "arbitrary set of methods"?

"Arbitrary" means:
Any kind of methods based on your need

So:

There is no fixed rule
You can write any methods with any logic

Example:

```
public class Calculator {

   public int add(int a, int b) {
      return a + b;
   }

   public int multiply(int a, int b) {
      return a * b;
   }
}
```

This is a Bean/Component because:

Don't distribute the entire application functionality into one single class
rather break down into several smaller components that encourages re-usability

If a class cannot complete by itself and want to use some other class then it is said to be dependent on another class

```
class Restaurant {
   public int getPrice(String item) {
      if (item.equals("Burger")) {
         return 100;
      } else if (item.equals("Pizza")) {
         return 200;
      } else {
         return 50;
      }
   }
}

class Customer {
   Restaurant restaurant; // dependency

   Customer(Restaurant restaurant) {
      this.restaurant = restaurant;
   }

   void orderFood(String item) {
      int price = restaurant.getPrice(item);
      System.out.println("Ordered: " + item);
      System.out.println("Total Price: " + price);
   }
}
```

Spring is all about managing the dependency between these classes